GenAI for QA Automation

Instructor-style course notes

- Prompt engineering and AI test generation
- GitHub Copilot and Selenium frameworks
- MCP servers and agentic AI with Claude Code
- n8n workflows, API and data testing
- AI testing tools, offline LLMs, test artifacts

Prepared for Girish Kumar, QA Automation Engineer (ERP domain)

Notes are section-wise and follow the course lesson order.

Contents

1. Prompt Engineering Foundations 4

2. Optimizing AI Test Generation 8

3. GitHub Copilot for Test Automation 11

4. MCP: Model Context Protocol 14

5. Agentic AI and Claude Code Skills 18

6. Custom Agents, CI/CD and Docker 22

7. n8n: AI-Powered Workflow Automation 25

8. GenAI for API and Data Testing 28

9. AI-Powered Testing Tools in the Market 30

10. Offline (Local) LLMs 32

11. Generating Test Artifacts on the Fly 34

Appendix: Prompt Library, Glossary and Cheat Sheet 36

How to use these notes

These notes follow your course in the same order as the curriculum, then add the topics listed in your learning objectives (AI testing tools, offline LLMs, test artifacts). Each section starts with the lessons it covers, explains the idea in plain language, and ends with revision points you can re-read the night before an interview or a demo.

Colored boxes appear throughout. Learn to recognise them:

INSTRUCTOR TIP
Practical advice from experience: what works, what wastes time.

ERP / YOUR PROJECT
How the idea maps to ERP testing: contract lifecycle, approvals, data-heavy forms.

WATCH OUT
Common mistakes, risks, and things AI gets wrong.

REMEMBER
Core points worth memorising.

SAMPLE PROMPT
A prompt you can copy, adapt and reuse.

Course map

#	Section	What you will be able to do
1	Prompt engineering foundations	Write prompts that give predictable, usable output; manage tokens and context
2	AI-driven test generation	Produce test plans, cases, strategy and data with AI, and defend it to a PM
3	GitHub Copilot	Use Ask, Agent and Plan modes; improve a Selenium Java framework inside IntelliJ
4	MCP servers	Explain MCP; build agents for browser, SQL, API, files and Excel
5	Agentic AI and Claude Code skills	Package domain knowledge as skills; let an agent write, run and fix tests
6	Custom agents, CI/CD and Docker	Automate delivery pipelines with AI help
7	n8n workflows	Build AI-agent workflows that connect Sheets, Jira and chat
8	GenAI for API and data testing	Parse JSON, generate POJOs, API tests and SQL; knowledge check
9	AI-powered testing tools	Evaluate commercial and open-source AI testing tools
10	Offline (local) LLMs	Run and customise a private LLM for sensitive projects
11	Test artifacts on the fly	Generate plan, cases, data and bug reports from requirements
A	Prompt library, glossary, cheat sheet	Quick reference

1. Prompt Engineering Foundations

Lessons covered: Why prompt engineering matters; zero-shot prompting with constraints; practice scenarios and few-shot prompting; context and constraints; chain of thought; tokens; context window limit.

1.1 Why prompt engineering matters

A large language model (LLM) does one thing: it predicts the most likely next piece of text, given everything you have sent it. It has no idea what you secretly meant. The quality of the answer is therefore limited by the quality of the instruction. This is exactly like testing: a vague requirement produces vague tests, and a precise requirement produces precise tests.

Prompt engineering is the skill of writing instructions so the model returns what you need on the first or second attempt. The course calls it an art because it is iterative: you try, look at the output, tighten the instruction, and try again.

The six building blocks of a strong prompt

Block	Question it answers	Example
Role	Who should the AI act as?	You are a senior QA engineer testing an ERP contract module.
Task	What exactly must it do?	Write test cases for the contract amendment flow.
Context	What does it need to know?	Contract is created, approved, then amended. Amendment fields are partly disabled.
Constraints	What are the limits?	Maximum 12 test cases. No UI locators. Do not invent fields.
Format	How should the answer look?	Table: ID, Title, Steps, Expected Result, Priority.
Examples	What does good look like?	One sample test case in your team format.

Weak prompt versus strong prompt

Weak	Strong
Write test cases for login.	Act as a QA engineer. Write 10 test cases for the login page of a web ERP application (fields: username, password, Login button; account locks after 5 failed attempts). Cover positive, negative and boundary cases. Output as a table with ID, Scenario, Steps, Expected Result, Priority (High/Medium/Low).

REMEMBER
Prompt = Role + Task + Context + Constraints + Format (+ Examples). If an answer is poor, the fix is nearly always more context or a tighter constraint, not a longer prompt.

1.2 Zero-shot prompting and adding constraints

Zero-shot means you give the instruction with no examples. The model relies only on what it learned in training. It works well for common tasks (summarise, explain, classify) and is the fastest way to start. Its weakness is inconsistency: the style, length and structure can vary between runs. Constraints are how you steer a zero-shot prompt.

Constraint type	What it controls	Example wording
Length	How much text	Answer in under 150 words.
Format	Structure	Return JSON with keys id, title, steps.

Constraint type	What it controls	Example wording
Scope	What is in or out	Only functional tests. Exclude performance and security.
Negative	What to avoid	Do not assume fields that are not listed.
Audience	Tone and depth	Explain for a non-technical project manager.
Quantity	How many items	Exactly 8 test cases: 3 positive, 3 negative, 2 boundary.
Source rule	Honesty	If information is missing, list it under Assumptions instead of guessing.

WATCH OUT
Constraints can contradict each other (for example, 'be very detailed' and 'under 50 words'). The model will silently pick one. Read your prompt once as if you were the model.

INSTRUCTOR TIP
The last row above is the single most useful constraint for QA work. It turns invented requirements into visible assumptions you can confirm with the business analyst.

1.3 Practice scenarios and few-shot prompting

Few-shot prompting means you include a small number of examples (usually 2 to 5) inside the prompt. The model copies the pattern. Use it whenever you need a consistent format, a specific naming convention, or a house style that the model cannot guess.

```
Convert each scenario into a Gherkin test. Follow the example style exactly.

Scenario: Approve a pending contract
Gherkin:
  Given a contract is in "Pending" status
  When the user clicks "Approve" and confirms
  Then the contract status is "Approved"

Scenario: Cancel an approved amendment
Gherkin:
```

Tips for good examples

- Keep every example in exactly the same format; the model copies inconsistencies too.
- Include at least one negative or edge example so it does not only produce happy paths.
- Use real wording from your project (field names, status names) so output matches your application.
- Two to five examples are normally enough. More examples cost tokens and add little.

Practice scenarios to try

Scenario	Technique	What to check in the output
Turn a one-line bug note into a full bug report	Zero-shot + format constraint	Steps, expected, actual, severity all present
Convert manual steps into Gherkin	Few-shot	Same Given/When/Then style as your examples
Suggest test data for a form with 6 fields	Zero-shot + quantity constraint	Valid, invalid and boundary values
Rename messy locator variables	Few-shot (before and after pairs)	Naming convention followed
Summarise a long failure log	Zero-shot + length constraint	Root cause line and failing step identified

1.4 Improving responses with context and constraints

Context is everything the model needs to know that it cannot guess. Add context in layers, starting with the most important:

1. **Domain:** what kind of system this is (commodity contract management, ERP).
2. **Feature:** the module or user story under test and the business rules around it.
3. **Technology:** language, framework and libraries (Java, Selenium, Cucumber, TestNG).
4. **Existing material:** a sample of your current code or test cases so the output matches.
5. **Audience and purpose:** who reads the answer and what decision it supports.

Then iterate. Read the first answer like a reviewer, name exactly what is wrong, and ask for only that change: 'Keep everything, but make the expected results measurable.' Short corrections work better than rewriting the whole prompt.

> **ERP / YOUR PROJECT**
> Give the model your status model once (Pending, Approved, Amended, Cancelled) and the allowed actions per status. Almost every ERP test idea depends on it, and models never know it by default.

1.5 Chain of thought prompting

Chain of thought (CoT) means asking the model to work through a problem step by step before giving the final answer. For tasks with several reasoning steps, this raises accuracy because each step becomes part of the context for the next one. It also lets you inspect the reasoning and catch a wrong assumption early.

> **SAMPLE PROMPT**
> A test failed with 'element not clickable at point'. Think step by step: list the likely causes in order of probability, say what evidence would confirm each, then give the single most likely cause and the fix. Show your reasoning before the conclusion.

Where CoT helps QA

- Root cause analysis of flaky or failing tests.
- Designing tests with boundary value analysis or state transitions.
- Estimating effort and risk for a release.
- Reviewing code: 'walk through this method line by line and report defects'.

WATCH OUT
Newer reasoning models think internally even without being asked. The visible explanation is a useful summary, not proof. Always verify conclusions against the real logs or application.

1.6 What are tokens and why they matter

Models do not read words; they read tokens, small pieces of text (a whole short word, part of a long word, punctuation or a space). As a rule of thumb for English, one token is roughly four characters, or about three quarters of a word. Code, JSON, tables and non-English text usually use more tokens per word.

Why tokens matter	Effect
Cost	Paid APIs and many plans charge per input token and per output token.
Speed	More tokens in and out means slower responses.
Limits	Every model has a maximum number of tokens it can handle at once (the context window).
Quality	Overloaded prompts bury the important instruction among noise.

INSTRUCTOR TIP
Pasting a 5,000-line log or an entire JSON payload is rarely needed. Paste the failing section plus a few lines before it. You save tokens and get a sharper answer.

1.7 Context window limit and saving tokens

The context window is the model's working memory for one conversation. It holds, together: the system instructions, your messages, the model's earlier replies, any attached files, tool results, and the reply being written. When the total reaches the limit, older content is dropped, summarised, or the request fails. Even before the limit, very long contexts make models less reliable on details in the middle.

Practical ways to save tokens

Technique	How
One task per chat	Start a fresh conversation when the topic changes.
Reference, do not paste	In IDE tools point to a file (for example #file) instead of pasting its content.
Trim inputs	Send only the relevant code, log section, or JSON branch.
Constrain outputs	'Return only the changed method', 'give a diff', 'max 10 bullets'.
Summarise and restart	Ask for a summary of decisions, open a new chat, paste the summary.
Persistent instructions	Store standing rules in a project file (custom instructions, CLAUDE.md, skills) instead of repeating them.
Use structured formats	Tables and JSON are compact and easy to reuse.

REMEMBER
Tokens are the currency, the context window is the budget. Spend them on information the model truly needs.

SECTION 1 REVISION
- Prompt = role, task, context, constraints, format, examples.
- Zero-shot for speed, few-shot for consistency, chain of thought for reasoning.
- About 4 characters per token in English; code and JSON cost more.
- Fresh chat per task; reference files; ask for small outputs.

2. Optimizing AI Test Generation

Lessons covered: Talk with your project manager; generating a test plan; generating test cases; test strategy with shift-left testing; test data combinations; privacy and security of AI applications.

2.1 Talking to your project manager about AI

This lesson is about getting permission and trust. A PM cares about delivery speed, risk and cost. Present AI as an accelerator for the repetitive parts of testing, with the tester still accountable for quality.

PM concern	Your answer
Is it accurate?	Every AI artifact is reviewed by a tester before use. AI drafts, humans approve.
Is it safe?	No real customer data in prompts; approved tools only; see section 2.5.
What do we gain?	First drafts of plan, cases and data in minutes. Less time on boilerplate, more on exploratory and risk-based testing.
How do we prove it?	Pilot on one module. Measure time to produce artifacts, defects found, review rework.
What does it cost?	Tool licences versus hours saved. Start with a small pilot.

INSTRUCTOR TIP
Bring a before-and-after example from your own module. A real artifact beats any slide.

2.2 Generating a test plan from business requirements

Give the model the requirement document or user stories and ask for a plan in your organisation's template. A useful plan contains the following parts; ask for them explicitly.

Section	What it contains
Objective and scope	What is tested and what is explicitly out of scope
Test approach	Types of testing: functional, regression, integration, API, security, performance
Test environment and data	Environments, accounts, data needs, masking rules
Entry and exit criteria	When testing starts and when it can stop
Risks and mitigations	What could go wrong and how to reduce it
Deliverables and schedule	Artifacts, milestones, owners
Assumptions and open questions	Everything the AI could not confirm

SAMPLE PROMPT
Act as a QA lead. From the requirements below, produce a test plan with these sections: objective, scope (in/out), approach, environment and data, entry/exit criteria, risks with mitigation, deliverables, assumptions and open questions. Do not invent requirements; anything unclear goes under open questions. Requirements: [paste].

WATCH OUT
AI plans look complete even when they are generic. Check that scope, risks and criteria mention your actual modules, integrations and constraints.

2.3 Generating test cases from requirements

Ask the model to apply named test design techniques, otherwise it tends to produce obvious happy paths. Name the technique, the format and the coverage you expect.

Technique	Use it for	ERP example
Equivalence partitioning	Groups of inputs behaving the same	Valid and invalid quantity ranges
Boundary value analysis	Edges of a range	Quantity 0, 1, max, max+1
Decision table	Combinations of rules	Pricing type with payment terms with tax structure
State transition	Objects with a lifecycle	Contract: Pending, Approved, Amended, Cancelled
Error guessing	Experience-based risks	Duplicate submit, expired session, missing mandatory fields

SAMPLE PROMPT
From the user story below, write test cases using equivalence partitioning, boundary value analysis and state transition. Output a table: ID, Title, Preconditions, Steps, Expected Result, Type (Positive/Negative/Boundary), Priority. Include at least 30 percent negative cases. Add a traceability column linking each case to the requirement line. Story: [paste].

ERP / YOUR PROJECT
The contract lifecycle (create, approve, modify, amend, amendment cancel) is a state machine. Ask the model for a state transition table first, then generate cases from each transition, including illegal transitions such as modifying an approved contract.

2.4 Test strategy and shift-left testing

Shift-left means testing earlier and at lower, cheaper layers so defects are found when they are least expensive to fix. In the test pyramid, many fast unit tests sit at the bottom, fewer API and service tests in the middle, and a small number of slow end-to-end UI tests at the top.

Unit tests (many, fast)	API / service tests	UI end-to-end (few, critical journeys)

A good AI exercise is to give the model your list of test cases and ask it to place each one in the cheapest layer that can still catch the defect, with a reason.

Test case	Best layer	Reason
Quantity must be greater than zero	Unit / API	Pure validation rule, no UI needed
Approved contract cannot be edited	API	Business rule enforced by the service
Approve button appears only for authorised role	UI + API	Needs permissions and display check
Create, approve and amend a contract end to end	UI	Critical journey across screens

SAMPLE PROMPT
Here is my test case list: [paste]. Classify each into Unit, API or UI using shift-left thinking. Give a one-line reason, then summarise the proportion per layer and recommend which UI tests could move down.

INSTRUCTOR TIP
Review the result with developers. The AI does not know which rules are really enforced in the backend; the team does.

2.5 Generating test data combinations

Test data is a strong use of AI because the model can enumerate combinations quickly. Ask for the technique and the output format together.

- **Equivalence and boundary values** for each field.
- **Pairwise (all-pairs) combinations** when many fields have several options; covers every pair of values with far fewer rows than all combinations.
- **Output as JSON or CSV** so it plugs straight into your data-driven framework.
- **Synthetic data only.** Never paste production records.

SAMPLE PROMPT
Generate test data for the contract form. Fields and options: Pricing (FLAT PRICE, NPE, PTBF); Inco terms (DWH, DDP, DEQ); Currency (EUR, USD, UGX); Fixation by (Seller's call, Buyer's call). Produce a pairwise set as JSON array, one object per test case, with a testCaseId.

ERP / YOUR PROJECT
This matches a data-driven JSON design like your contract test data. Always check generated values against real master data (valid counterparties, ports, currencies); the AI cannot know what exists in your system.

2.6 Privacy and security of AI applications

Using AI on a real project exposes new risks. Companies are adapting by writing AI usage policies, buying enterprise plans that do not train on company data, and in some cases running models privately.

Risk	Example	Control
Data leakage	Pasting customer data, contracts or credentials into a public chatbot	Mask or synthesise data; approved enterprise tools only
Hallucination	Invented field names or business rules	Review against requirements; ask for assumptions
Prompt injection	Hidden instructions inside a web page or document an agent reads	Least-privilege tools, human approval, trusted sources
Insecure generated code	Hard-coded secrets, weak validation	Code review, secret scanning, static analysis
Licence and IP	Generated code resembling licensed code	Follow company policy on AI-generated code
Over-trust	Shipping AI output unreviewed	Human sign-off on every artifact

REMEMBER
Never put passwords, API keys, personal data or production records into a prompt. If the data is sensitive, use an approved enterprise deployment or an offline model (section 10).

SECTION 2 REVISION
• AI drafts plans, cases, strategy and data; humans review and approve.
• Use named techniques (BVA, EP, state transition, pairwise) in prompts.
• Shift-left: push each test to the cheapest layer that catches the defect.
• Synthetic data only; secrets never in prompts.

3. GitHub Copilot for Test Automation

Lessons covered: Introduction and features; Ask and Agent modes in VS Code; Planning mode; installing on IntelliJ, PyCharm and Eclipse; Copilot plugin for Selenium Java frameworks in IntelliJ.

Interface names and menu paths change between releases. Use these notes for the concepts and check the current GitHub documentation for exact labels.

3.1 What Copilot offers

Feature	What it does
Inline suggestions	Completes the line or method as you type, based on surrounding code
Chat	Ask questions about code, errors and design in a side panel or inline
Agent mode	Plans multi-step work, edits several files, runs commands, reacts to errors
Custom instructions	A file with your coding standards that Copilot reads for every request
Context controls	Attach files or the whole workspace so answers match your code
Model choice	Pick among available models for different tasks

3.2 Ask mode versus Agent mode in VS Code

	Ask mode	Agent mode
Purpose	Questions, explanations, ideas	Carry out a task
Changes files?	No	Yes, across several files
Runs commands?	No	Yes (terminal, build, tests), usually with your approval
Iterates on errors?	No	Yes: reads the error, fixes, retries
Best for	'Why does this wait fail?', 'Explain this class'	'Add a page object for the amendment screen and a step definition'
Risk	Low	Higher: review every change

- Start in Ask mode to understand the problem. Switch to Agent mode when you know what you want built.
- Commit or stash your work first so you can review or discard the agent's changes as a diff.
- Give the agent a clear goal, constraints and a way to verify (for example 'run the amendment tests and fix compile errors').

WATCH OUT
Agents can run terminal commands. Read each command before approving it, and never auto-approve on a machine with access to production systems.

3.3 Planning mode: research and design before coding

Planning mode makes Copilot investigate the code base and propose a step-by-step plan without changing files. You review the plan, edit it, and only then hand it to the agent to implement. It is ideal for larger changes where a wrong first move wastes time.

1. Describe the goal: 'Replace Thread.sleep with explicit waits across the framework.'

2. Copilot reads the project and lists affected files, risks and an ordered plan.
3. You adjust the plan ('skip the utility package, do pages first').
4. Switch to Agent mode and ask it to execute the approved plan step by step.

> **INSTRUCTOR TIP**
> Planning is where you catch wrong assumptions cheaply. A five-minute plan review often saves an hour of undoing agent edits.

3.4 Installing Copilot in IntelliJ and PyCharm

1. Open Settings, then Plugins, then Marketplace.
2. Search for 'GitHub Copilot' and click Install.
3. Restart the IDE when prompted.
4. Sign in to GitHub from the Copilot icon; a device code appears. Enter it on the GitHub page and authorise.
5. Confirm that your GitHub account has an active Copilot licence (individual or organisation).
6. Open the Copilot chat panel and test with a simple question about the open file.

IntelliJ IDEA is the common choice for Java frameworks; PyCharm is the same plugin for Python projects.

3.5 Installing Copilot in Eclipse

1. Go to Help, then Eclipse Marketplace.
2. Search for 'GitHub Copilot' and install it.
3. Restart Eclipse and sign in with your GitHub account when asked.
4. Open the Copilot chat view and test on a Java class.

Feature coverage in Eclipse and the JetBrains IDEs can differ from VS Code (for example the newest agent features arrive in VS Code first). Check release notes if a feature seems missing.

3.6 Using Copilot on a Selenium Java framework in IntelliJ

This was the longest lesson. The goal is to use AI to build and improve a framework while keeping to your standards.

Step 1: write down your standards once

Create a custom instructions file (commonly **.github/copilot-instructions.md**) so Copilot follows your conventions without being reminded.

```
# Automation standards
- Language: Java 17, Selenium 4, Cucumber BDD, TestNG.
- Page Object Model: one class per page, no assertions inside page classes.
- Waits: explicit waits (WebDriverWait) only. Never Thread.sleep.
- Locators: prefer id / data-test attributes, then CSS. Avoid long XPath.
- Naming: methods start with a verb (clickApprove, selectPricing).
- Logging: use the framework logger, no System.out.println.
- Test data: read from JSON test data; never hard-code values in steps.
```

Step 2: typical tasks to give Copilot

Task	Mode	Example prompt
Explain unfamiliar code	Ask	Explain what this method does and where it can fail.
Create a page object	Agent	Create a page class for the Shipping Instruction screen with methods for the listed fields.
Write step definitions	Agent	Add Cucumber step definitions for the new feature file, reusing existing helpers.
Remove duplication	Plan then Agent	Find repeated dropdown-selection code and propose one reusable method.
Fix flaky waits	Agent	Replace Thread.sleep with explicit waits and run the smoke tests.
Generate unit tests for utilities	Agent	Write TestNG tests for the date and JSON helper classes.

Step 3: review checklist for AI-written automation code

- Locators are stable and unique; no brittle absolute XPath.
- Waits are explicit; no fixed sleeps.
- Assertions check business outcomes, not just that an element exists.
- No duplicated logic; existing utilities reused.
- No hard-coded data or credentials.
- The code compiles and the affected tests actually pass when run.

ERP / YOUR PROJECT
ERP front ends built with component libraries (for example dropdowns, calendars and tables from a UI kit) need special handling. Ask Copilot to create one reusable helper per component type instead of repeating click-and-type logic in every step.

SECTION 3 REVISION
• Ask mode explains; Agent mode changes files; Plan mode designs first.
• Put standards in a custom instructions file once.
• Commit before agent runs; review every diff.
• Verify by running the tests, not by reading the code only.

4. MCP: Model Context Protocol

Lessons covered: What is MCP and why it makes an LLM powerful; resources to download; browser agent with Playwright/Selenium MCP; debugging MCP configuration failures; justifying MCP to your PM; SQL database agent; API testing and local file system agent; Excel agent; Playwright MCP in VS Code.

4.1 What is MCP?

An LLM on its own can only read and write text. It cannot click a button, query a database or open a file. The Model Context Protocol (MCP) is an open standard that lets AI applications connect to external tools and data in one uniform way. A common analogy is a USB-C port: one connector that works with many devices. Without a standard, every AI app would need a custom integration for every tool.

Architecture

Host app (VS Code, Claude, IDE)	MCP client (inside the host)	MCP server (one per tool)	Real system (browser, DB, files, API)

Part	Role
Host	The application you talk to; it contains the LLM and decides what the model may use
Client	Maintains the connection to one MCP server on behalf of the host
Server	A small program that exposes capabilities of a system in a standard way
Tools	Actions the model can call (click, run query, read file)
Resources	Data the model can read (files, records, documents)
Prompts	Reusable prompt templates the server offers
Transport	How client and server talk: locally over standard input/output (stdio) or remotely over HTTP

How the loop works

1. You give a goal in plain language.
2. The model sees the list of tools each connected server offers and chooses one.
3. The host asks for your approval (if required) and calls the tool on the server.
4. The server performs the action and returns the result.
5. The model reads the result and decides the next step, repeating until the goal is done.

REMEMBER
MCP turns an LLM from something that talks into something that acts. The model supplies reasoning; MCP servers supply hands and eyes.

WATCH OUT
An MCP server runs with real permissions. Install only servers you trust, give them the minimum access they need, and treat their output as untrusted input (it can contain prompt injection).

4.2 Resources to download

A short lesson that points to the course downloads (configuration files and samples). Keep them in one folder, and note which VS Code or Claude config each file belongs to.

4.3 An agent that automates a browser with Playwright or Selenium MCP

A browser MCP server exposes tools such as open page, click, type, select, take snapshot and read text. The Playwright MCP server describes the page to the model using a structured snapshot of the page's accessibility tree (roles, names, states) rather than relying on screenshots alone. That makes element identification faster and more reliable. Community Selenium MCP servers work on the same principle with a Selenium back end.

This is what the course calls codeless browser automation: you describe the scenario in English and the agent drives the browser.

> **SAMPLE PROMPT**
> Open the application URL, log in with the test user, create a new purchase contract with pricing FLAT PRICE, save it, and confirm the status shows Pending. Report each step and stop if any element is missing.

Good for	Not good for
Exploring an application quickly	Replacing a stable regression suite
Drafting a first version of a test script	Fast, repeatable, cheap nightly runs
Reproducing a bug from a description	Tests that must be fully deterministic
Checking a flow on a new build	Large data-driven runs (slow, token cost)

> **INSTRUCTOR TIP**
> Use the agent to explore and to generate the first draft of a script, then keep the script in your normal framework. You get AI speed during authoring and framework reliability during runs.

4.4 Debugging MCP configuration failures

Symptom	Likely cause	What to do
Server never starts	Node.js missing or too old	Install a current Node.js LTS; check node -v
'command not found'	npx or the program is not on PATH	Use the full path or fix the PATH; restart the host app
Config ignored	JSON syntax error or wrong key name	Validate the JSON; VS Code uses a 'servers' key, some other hosts use 'mcpServers'
Tools do not appear	Server crashed after start	Open the host's MCP output/log panel and read the error
Works in terminal, fails in app	Different environment variables	Set the env block explicitly in the config
Browser does not open	Browser binaries not installed	Run the server's browser install command
Intermittent failures	Old cached package	Clear the npx cache or pin a version

```
// VS Code: .vscode/mcp.json
{
  "servers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    }
  }
}
```

INSTRUCTOR TIP
When a server fails, run its command by hand in a terminal first. If it fails there, the problem is the server or your machine, not the AI tool.

4.5 Justifying MCP to your project manager

Question	Answer
What problem does it solve?	Connects AI to our real tools (browser, DB, files) without custom code per tool
Business value	Faster exploration and script drafting; less manual data lookup
Cost	Open-source servers are free; main cost is model usage and setup time
Risk	Tool access must be scoped: read-only DB users, limited folders, test environments only
Next step	Two-week pilot on one module with defined success measures

4.6 SQL database agent

A database MCP server lets the model inspect the schema and run queries. The agent can turn a question like 'which approved contracts have no shipping instruction?' into SQL, run it, and explain the result. In testing it is used for data validation: compare what the UI shows with what the database stores, and find test data.

- Use a **read-only** database account.
- Connect to a test or QA database, never production.
- Ask the agent to show the SQL before it runs, and to limit rows.
- Give it the table and column descriptions; schema names alone are often cryptic.

SAMPLE PROMPT
Using the connected database, list the 10 most recent contracts in Approved status with their counterparty and total quantity. Show the SQL first, use read-only queries only, and limit the result to 10 rows.

ERP / YOUR PROJECT
Joins across contract, product and commission tables are exactly where a model saves time, and also where it can silently join the wrong key. Check the join conditions yourself.

4.7 API testing and local file system agent

Combining an HTTP/API tool with a file system server allows an agent to read test data from a local folder, call an endpoint, compare the response with expected values, and write a result file. Restrict the file server to one folder.

1. Place request samples and expected responses in the allowed folder.
2. Ask the agent to read a sample, call the endpoint, and validate status code, schema and key fields.
3. Ask it to write a result summary file and list any mismatches.

WATCH OUT
Do not point a file server at your whole drive. A mistaken or injected instruction could read or overwrite sensitive files.

4.8 Excel agent

An Excel MCP server can read and write workbook cells. Typical uses are reading test data rows, writing execution results back, and building a defect summary sheet.

- Keep a backup copy; agents can overwrite cells.
- State the sheet name, columns and range in the prompt.
- Prefer writing results to a new sheet or file so the source data stays intact.

4.9 Playwright MCP in VS Code: generating tests

1. Add the Playwright MCP server in the VS Code MCP configuration (see 4.4) and start it.
2. Open Copilot chat in Agent mode and confirm the browser tools are listed.
3. Describe the scenario. The agent explores the application and observes real elements.
4. Ask it to write a Playwright test from what it did, using role- or label-based locators.
5. Run the test, review it, and refactor into your page objects and fixtures.

INSTRUCTOR TIP
Ask the agent to add assertions for the business outcome (status text, message) and not only that navigation succeeded.

SECTION 4 REVISION
• MCP = open standard to connect LLMs to tools and data (host, client, server).
• Servers expose tools, resources and prompts.
• Browser, SQL, API, file and Excel agents are all MCP servers plus a goal.
• Scope permissions tightly; debug by running the server command manually.

5. Agentic AI and Claude Code Skills

Lessons covered: Introduction to agentic AI and action plan; the skill system problem statement; installing Claude Code and Claude in Chrome and /init; knowledge skills versus agent skills; skill docs for the EventHub application; avoiding context bloat; agents creating test scenarios and test strategy; Playwright best-practice skills; running tests and fixing failures; coverage analysis.

5.1 What is agentic AI?

A chatbot answers. An agent pursues a goal. It plans, takes actions with tools, looks at the result, and keeps going until the goal is met or it needs help.

Goal	Plan	Act (use tools)	Observe result	Adjust / finish

Agent ingredient	Meaning
Model	The reasoning engine (the LLM)
Tools	What it can do: edit files, run commands, browse, query
Instructions	Rules, role, standards it must follow
Memory and knowledge	Project facts it can recall or look up
Loop	Repeats plan, act, observe until done

The problem agents solve in testing is repetitive multi-step work: read requirements, write tests, run them, read failures, fix, rerun. The action plan of this section is to give an agent the knowledge and the rules to do that reliably.

5.2 The problem skills solve

A general model does not know your product, your business rules or your team's standards. If you re-explain them in every chat you waste tokens and get inconsistent results. The Claude Code skill system packages that knowledge once, in files, so the agent can load what it needs, when it needs it.

5.3 Installing Claude Code and Claude in Chrome; the /init command

- **Claude Code** is a terminal-based agentic coding tool. Install it using the method on the official documentation page (installation options change over time), then run **claude** inside your project folder and sign in.
- **Claude in Chrome** is a browser extension that lets Claude work in your Chrome browser, useful for exploring a web application as part of a task.
- **/init** scans the project and creates a **CLAUDE.md** file: a project memory with how to build and test, the structure, and conventions. Claude reads it at the start of each session. Review and edit it; it is your standing briefing for the agent.

> **INSTRUCTOR TIP**
> Keep CLAUDE.md short and accurate. A bloated CLAUDE.md costs tokens in every session.

5.3a Knowledge skills versus agent skills

The course separates two kinds of skill. Both live in the project as folders, each with a **SKILL.md** file whose name and description tell the agent when to use it.

	Knowledge skill	Agent skill
What it holds	Facts: domain rules, glossary, screens, data, conventions	Procedures: how to do a task step by step
Answers	'What is true in this application?'	'How do I do this job?'
Example	EventHub domain document	Write Playwright tests following our rules
Changes when	The product changes	The way of working changes
Used for	Accuracy and context	Repeatable workflows

They work best together: the agent skill describes the workflow, and it points to the knowledge skill for the facts.

```
.claude/skills/
  eventhub-domain/            (knowledge skill)
    SKILL.md                  short summary + links
    modules.md  roles.md  business-rules.md  test-data.md
  playwright-test-writer/     (agent skill)
    SKILL.md                  steps: read scenario, write test, run, fix
    best-practices.md
```

```
---
name: eventhub-domain
description: Business rules, roles and modules of the EventHub application.
  Use when writing or reviewing tests for EventHub.
---
# EventHub domain
Overview in 5 lines. For details load: modules.md, roles.md, business-rules.md.
```

5.4 Creating skill docs for the EventHub application

EventHub is the demo application used in the course. The lesson shows how to turn an application into documents an agent can use. The same method applies to your own product.

Document	Contents
Overview	Purpose, users, main journeys, URLs, environments
Modules and screens	Each screen, its fields and actions
Roles and permissions	Who can do what
Business rules	Validations, status changes, calculations, limits
Test data	Valid sample data, accounts, boundary values
Known behaviours	Quirks, bugs, areas that change often

Design principles

- One topic per file; small files are cheap to load.
- Write facts, not opinions, in short plain sentences.
- Use consistent headings and field names that match the real UI.
- Include concrete examples (inputs and expected outputs).
- Keep a short index so the agent knows which file to open.

ERP / YOUR PROJECT
For your ERP, write a status-and-action matrix (status versus allowed actions) and a rules file per module. These two documents carry most of the value.

5.5 Avoiding context bloat with smart references

Loading everything into context is the most common mistake. Instead use progressive disclosure: keep the main SKILL.md short, and reference detail files that the agent opens only when the task needs them.

Do	Do not
Short SKILL.md with a precise description and links to detail files	One giant file holding every rule
Reference files by path and ask the agent to read only what it needs	Paste whole documents into the prompt
Write descriptions that state when to use the skill	Vague descriptions that load the skill for everything
Check context usage regularly (Claude Code has a context view)	Assume the budget is unlimited
Start a new session for a new task	Run one session for the whole day

5.6 Agent creating test scenarios from the domain document

With a domain knowledge skill in place, the agent can derive scenarios that match your application instead of generic ones.

SAMPLE PROMPT
Read the EventHub domain skill. For the booking module, produce test scenarios grouped by feature: positive, negative, boundary and role-based. For each scenario give the rule it verifies and the source document section. List any rule you could not find.

- Insist on a source reference per scenario; it makes review fast and exposes invented rules.
- Review scenarios with a business analyst before automating.

5.7 Agent creating a test strategy to push tests into layers

Here the agent applies shift-left thinking (section 2.4) using real knowledge of the application. It assigns each scenario to unit, API or UI and justifies the choice. Ask it to include a short summary of how many tests go to each layer and which UI tests are the true critical journeys.

5.8 Playwright best-practice skills and an agent that writes tests

An agent skill can encode your team's Playwright standards so every generated test follows them.

Best practice	Why
Locate by role, label or text first	Resilient to markup changes
Rely on auto-waiting; no fixed sleeps	Faster and less flaky
Use web-first assertions that retry	Stable checks
Independent tests with their own data	Run in any order, in parallel
Page objects or fixtures for reuse	Less duplication

Best practice	Why
Capture trace, screenshot and video on failure	Faster diagnosis
Tag tests (smoke, regression)	Selective runs

Workflow for the agent: read the scenario, load domain and best-practice skills, write the test, run it, and report.

5.9 Agent running tests and fixing failures using the domain documents

Run tests	Read failure	Classify cause	Check domain doc	Fix test or report bug	Rerun

Cause category	Action
Test issue (wrong locator, timing)	Fix the test
Test data issue	Fix data or setup
Environment issue	Report; do not change the test
Application defect (domain doc says the app is wrong)	Raise a bug report; do not hide it

WATCH OUT
An agent under pressure to make tests pass may weaken assertions or delete checks. Add an explicit rule to the skill: never weaken an assertion to make a test pass; if the application disagrees with the domain document, report a defect.

5.10 Goal-oriented agentic solution: coverage analysis with a report

Give the agent a goal instead of steps: 'analyse how well our tests cover the requirements and produce a report.' The agent lists requirements, maps existing tests to them, finds gaps and ranks them by risk.

Report section	Content
Coverage matrix	Requirement versus test cases that cover it
Gaps	Requirements with no or weak tests
Risk ranking	Which gaps matter most (impact and likelihood)
Recommendations	New tests to add and layer to put them in

SECTION 5 REVISION
- Agent = model + tools + instructions + memory in a loop.
- Knowledge skills hold facts; agent skills hold procedures.
- Short SKILL.md, detail files loaded on demand: avoids context bloat.
- Agents classify failures before fixing; never weaken assertions.

6. Custom Agents, CI/CD and Docker

Lessons covered: GitHub Copilot custom agents; Copilot cloud agents; CI/CD with GitHub Actions and pushing to a remote repository; CI/CD demo with a Claude Code prompt; containerising tests with Docker; manager and QA conversation about DevOps.

6.1 GitHub Copilot custom agents

A custom agent is a specialised assistant with its own instructions, allowed tools and output format. You define it in a file in the repository (currently an .agent.md file under .github/agents; older versions used 'chat modes'), so the whole team shares it.

Example agent	Job	Key instructions
Test reviewer	Review new test code	Check locators, waits, assertions, duplication; list findings by severity
Bug report writer	Turn notes into a bug report	Use the team template; ask for missing details
Locator fixer	Repair failing locators	Propose the most stable locator; never change assertions

- One responsibility per agent.
- Give only the tools it needs.
- Specify the exact output format so results are consistent.

6.2 Copilot cloud agents

A cloud agent runs on GitHub's infrastructure rather than on your laptop. You assign it an issue or task; it works in the background in its own environment, commits to a branch and opens a pull request for you to review. Good tasks are small and well defined: add tests for a utility, update a dependency, fix a clear bug.

> **WATCH OUT**
> Treat its pull request like a colleague's: read the diff, run the tests, and review for security. Vague issues produce vague changes.

6.3 CI/CD with GitHub Actions and pushing code to a remote repository

Continuous Integration (CI) runs your tests automatically on every change. Continuous Delivery (CD) automates getting a tested build to an environment. In GitHub Actions a **workflow** (a YAML file in .github/workflows) is triggered by an **event** (push, pull request, schedule) and runs **jobs** made of **steps** on a **runner**.

```
name: Regression tests
on:
  push:
    branches: [main]
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
      - run: mvn -B test
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: test-reports
          path: target/
```

Pushing code to a remote repository

```
git init
git add .
git commit -m "Add regression suite"
git remote add origin https://github.com/<org>/<repo>.git
git push -u origin main
```

WATCH OUT
Store credentials, URLs and tokens in GitHub Secrets, never in the repository or the workflow file.

6.4 CI/CD with a single Claude Code prompt

The demo shows that one clear prompt can create the workflow file, commit and push. It is a fast way to get a first pipeline, but you must review the generated YAML and the git commands.

SAMPLE PROMPT
Create a GitHub Actions workflow that runs my Maven tests on every push and pull request using Java 17, uploads the test report as an artifact, and takes the base URL from a repository secret named BASE_URL. Show me the file before committing.

6.5 Containerising tests with Docker

Docker packages the test code and its runtime into an image so it behaves the same on your laptop, a colleague's machine and the CI server. That removes 'it works on my machine' problems.

```
FROM maven:3.9-eclipse-temurin-17
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
CMD ["mvn", "-B", "test"]
```

```
docker build -t ui-tests .
docker run --rm -v "$(pwd)/target:/app/target" ui-tests
```

- Run browsers in headless mode inside containers.
- Mount a volume so reports are saved outside the container.

- Pass environment-specific values with environment variables, not baked into the image.

6.6 Manager and QA conversation about DevOps solutions

Manager asks	QA answers
Why automate the pipeline?	Every change gets tested within minutes; defects are found early
What will it cost?	Mostly setup time and runner minutes; saves repeated manual regression
What if tests are flaky?	We track flaky rate and fix or quarantine them; flaky tests are treated as bugs
How do we measure success?	Pipeline duration, pass rate, flaky rate, time to detect defects

SECTION 6 REVISION
• Custom agents: single-purpose, shared, with defined tools and output.
• Cloud agents work in the background and deliver pull requests for review.
• GitHub Actions: workflow, event, job, step, runner; secrets stay in Secrets.
• Docker: same environment everywhere; mount volumes for reports.

7. n8n: AI-Powered Workflow Automation

Lessons covered: What is n8n and business process automation; n8n with AI agents; agent that reads a Google Sheet and finds bugs in New status; setting up Jira Cloud; plugging Jira into the agent and building an end-to-end workflow; public chat interface through a webhook.

7.1 What is n8n?

n8n is a workflow automation platform. You connect applications visually: a **trigger** starts a workflow, and a chain of **nodes** processes data and calls other services. It can be self-hosted or used as a cloud service, and it has hundreds of ready integrations (Google Sheets, Jira, Slack, email, databases, HTTP).

Concept	Meaning
Workflow	A complete automation made of connected nodes
Trigger node	What starts it: schedule, webhook, new row, chat message, manual
Action node	Does something: read a sheet, create an issue, send a message
Credentials	Stored logins/tokens for each connected service
Execution	One run of the workflow; you can inspect the data at every node
Expression	A small snippet that picks values from earlier nodes

Business process automation means removing manual hand-offs: instead of someone copying new bugs from a spreadsheet to Jira, a workflow does it.

7.2 n8n with AI agents

n8n includes an **AI Agent** node. It connects to a chat model (for example an OpenAI or Anthropic model), optional memory, and a set of **tools** (other n8n nodes such as Google Sheets or Jira). The agent decides which tool to call to reach the goal in its instructions, so the workflow is no longer a fixed line; it adapts.

Trigger (chat / schedule)	AI Agent node	Model + memory + tools	Result / action

Fixed workflow	AI agent workflow
Same steps every time	Agent chooses steps based on the input
Predictable, cheap	Flexible, costs model tokens, needs guardrails
Best for stable rules	Best for judgement, summarising, classification

> **INSTRUCTOR TIP**
> Use plain workflow logic wherever the rule is clear, and an agent only for the parts that need judgement. It is cheaper and easier to test.

7.3 Agent that reads a Google Sheet and finds bugs in 'New' status

1. Create a Google Sheet with columns such as ID, Title, Description, Severity, Status.
2. In n8n add a Google Sheets credential (OAuth) and a Google Sheets node (or use it as an agent tool).
3. Filter rows where Status equals New.
4. Pass those rows to the AI Agent with instructions: check completeness, set a suggested severity, flag duplicates, and write a short summary.

5 Send the result to a chat channel or the next step.

SAMPLE PROMPT
You review bug reports. For each row with Status = New, check that title, steps and expected result are present. Output JSON with id, missing_fields, suggested_severity and a one-line reason. Do not change any data.

7.4 Setting up Jira Cloud

1 Create a free Jira Cloud site with an Atlassian account.

2 Create a project (for example a Scrum or Kanban software project) and note its key.

3 Check the issue types available (Bug, Task, Story).

4 Create an API token from your Atlassian account security settings.

5 In n8n create a Jira credential using your site URL, email and the API token.

WATCH OUT
An API token acts as you. Store it only in n8n credentials and revoke it if exposed.

7.5 Jira tool plus agent: an end-to-end workflow

Add the Jira node as a tool for the agent so it can create issues. The complete business case:

Read New rows from Sheet	Agent validates and enriches	Agent creates Jira Bug	Sheet status updated	Summary sent

- The agent maps sheet fields to Jira fields (summary, description, priority).
- Ask it to avoid duplicates by searching Jira first.
- Update the sheet row afterwards with the Jira issue key, so the row is not processed twice.
- Start with a manual trigger and one row; schedule it only after it is stable.

WATCH OUT
Agents can create wrong or duplicate issues. Add a limit on items per run, log every action, and begin in a test Jira project.

7.6 Public chat interface through a webhook

A **webhook** is a URL that starts a workflow when something calls it. n8n's chat trigger can expose the agent through a chat page or a URL that a website can call, so other people can talk to your AI workflow without n8n access.

1 Add a Chat Trigger (or Webhook) node at the start of the workflow.

2 Switch the workflow to **Active** so the production URL works.

3 Copy the production URL and share the chat page or call it from your own front end.

4 Test with a message and inspect the execution.

WATCH OUT
A public URL means anyone can use it and spend your model tokens. Add authentication or rate limits for anything beyond a demo, and never expose tools that change data to anonymous users.

SECTION 7 REVISION

- n8n: triggers + nodes; AI Agent node uses a model, memory and tools.
- Fixed logic where rules are clear, agents where judgement is needed.
- Sheet to agent to Jira is the classic QA example; update the source to avoid reprocessing.
- Public webhooks need authentication and limits.

8. GenAI for API and Data Testing

Lessons covered: Parsing JSON responses; generating POJO classes and Java methods; API tests in Cypress and Playwright from a contract; complex SQL queries; knowledge check on MCP, LLM and AI agents.

8.1 Parsing JSON responses with AI

Paste a sample response (with sensitive values removed) and ask the model to explain the structure, list the fields and types, and propose validations.

SAMPLE PROMPT
Here is a JSON response from the contracts API. Describe each field, its type, whether it looks mandatory, and suggest 10 assertions (status code, schema, business rules). Then write RestAssured code to check them. JSON: [paste sample].

INSTRUCTOR TIP
Give two or three samples (success, error, empty list). One sample hides optional fields.

8.2 Generating POJO classes and Java methods

A POJO (plain old Java object) maps a JSON structure so you can deserialise a response into typed objects instead of digging through strings. AI is fast at generating them from a sample.

```
// Generated from sample JSON (Jackson)
@JsonIgnoreProperties(ignoreUnknown = true)
public class ContractResponse {
    private String contractNo;
    private String status;
    private List<ProductLine> products;
    // getters and setters
}

ContractResponse r = given().when().get("/contracts/{id}", id)
        .then().statusCode(200).extract().as(ContractResponse.class);
assertEquals("Approved", r.getStatus());
```

- Ask for @JsonIgnoreProperties(ignoreUnknown = true) so new fields do not break old tests.
- Ask for nested classes and lists to be modelled, and for correct number types (BigDecimal for money).
- Check date formats and nullable fields; models often guess these wrongly.
- Compile and run it; do not trust code that has not been executed.

ERP / YOUR PROJECT
Use BigDecimal for quantities and prices in ERP responses. Floating point types cause rounding mismatches that look like application defects.

8.3 API tests in Cypress and Playwright from an API contract

An API contract (OpenAPI/Swagger) describes every endpoint, input and output. Give it to the model and ask for tests per endpoint: success, validation errors, authorisation, and schema checks.

SAMPLE PROMPT
From this OpenAPI definition generate Playwright API tests (TypeScript) for POST /contracts. Cover: valid request, each missing mandatory field, wrong data type, unauthorised call, and response schema. Use request fixtures and keep test data in a separate object.

Check	Why
Status code	Correct success and error codes
Response schema	Fields, types, required properties
Business rules	Calculated values, status transitions
Negative inputs	Missing, wrong type, out of range
Authentication and authorisation	Anonymous and wrong-role calls rejected
Idempotency and duplicates	Repeat submit does not create two records

8.4 Complex SQL queries with AI

Provide the table definitions and a plain-language question; ask for the query and an explanation. This is useful for validating data after UI or API actions.

SAMPLE PROMPT
Tables: contract(id, contract_no, status, counterparty_id), product_line(id, contract_id, qty, price), counterparty(id, name). Write a read-only SQL query that returns contract_no, counterparty name and total value (qty x price) for Approved contracts with total above 100000, sorted descending. Explain the joins.

WATCH OUT
Always read the query before running it. Check join keys, filters, and anything that changes data (UPDATE, DELETE). Use a read-only account on a test database.

8.5 Knowledge check: MCP, LLM and agents

Question	Answer
What is an LLM?	A model trained on text that predicts the next token; generates language and code
What is MCP?	A standard protocol connecting AI apps to external tools and data
What is an AI agent?	A model with tools and instructions that plans and acts in a loop toward a goal
Difference between a chatbot and an agent?	A chatbot replies; an agent takes actions and iterates
Why does context matter?	The model only knows what is in its context window and training
What is a hallucination?	Confident output that is not true or not grounded in your data
What reduces hallucination?	Grounding in your documents, clear constraints, verification, human review

SECTION 8 REVISION
- AI generates POJOs, RestAssured and Playwright API tests, and SQL fast.
- Always compile, run and review; check types, nulls and joins.
- Use samples of several response types; BigDecimal for money.

9. AI-Powered Testing Tools in the Market

Lessons covered: Learning objective: overview of AI-powered testing tools and their capabilities. The detailed product list was not in your curriculum text, so these notes describe categories and evaluation criteria. Product names are examples only; features and pricing change quickly, so verify them on the vendor sites.

9.1 What AI adds to testing tools

Capability	What it does	Value
Self-healing locators	When a locator breaks, the tool finds the element using other attributes and updates the test	Less maintenance
Natural-language test authoring	Write steps in English; the tool executes or converts to scripts	Faster authoring, wider team access
Autonomous exploration	Agent crawls the application and finds issues or generates tests	Quick coverage of new areas
Visual validation	Compares screens by perceived appearance, not pixels	Catches layout defects
Test generation	Creates tests from requirements, recordings or traffic	Faster first drafts
Failure analysis	Groups failures, suggests root cause	Faster triage
Test selection	Picks the tests most likely affected by a change	Shorter feedback
Test data generation	Creates realistic synthetic data	Safer and quicker data

9.2 Categories of tools (examples)

Category	Typical examples	Notes
AI-assisted test platforms	Testim, mabl, Functionize, Katalon	Low-code with self-healing and AI authoring
Natural-language / codeless	testRigor, KaneAI and similar	Plain-English tests
Visual AI	Applitools	Visual comparison engines
Code assistants	GitHub Copilot, Claude Code, Cursor	Help engineers write framework code
Agentic browser tools	Playwright MCP and similar	Agent-driven exploration and script drafting
Workflow automation	n8n and similar	Connect AI with Jira, sheets, chat

WATCH OUT
This list is illustrative. I have not verified current features or pricing for these products; check each vendor before recommending anything.

9.3 How to evaluate a tool

Criterion	Questions to ask
Fit with application	Does it handle your UI components, canvas-like widgets, iframes and downloads?
Reliability	How often do AI-driven steps behave differently between runs?
Maintainability	Can tests be exported and reviewed, or are you locked in?
Security and data	Where does data go? Is there an enterprise or on-premise option?
Integration	CI/CD, Jira, reporting, existing framework
Cost	Licence, usage-based charges, training time
Skills	Can your team operate and debug it?
Proof	Run a pilot on your own application, not the vendor's demo

REMEMBER
Pilot every tool on a few of your hardest screens. Demo applications are always easy.

9.4 Limitations to be honest about

- AI-driven execution can be slower and less repeatable than a coded test.
- Self-healing can hide real defects when a changed element is a genuine bug.
- Usage-based pricing can grow quickly with large suites.
- Complex ERP business logic still needs human-designed tests and data.

SECTION 9 REVISION
• Tools help most with authoring, maintenance, triage and visual checks.
• Evaluate by pilot on your own hardest screens.
• Self-healing needs oversight so it does not mask defects.

10. Offline (Local) LLMs

Lessons covered: Learning objective: work with offline LLMs for full privacy and customise them for your project. This topic was in your learning objectives but not in the pasted lesson list, so these notes cover the standard approach. Tool names are examples; versions change.

10.1 Why run a model locally

Reason	Detail
Privacy	Prompts and files never leave your machine or network
Compliance	Meets rules that forbid sending code or data to external services
Cost	No per-token fees; you pay with hardware
Offline use	Works without internet
Control	Choose the model, version and settings; no sudden changes

Trade-off	Detail
Quality	Smaller local models are generally weaker than the largest hosted models
Hardware	Needs enough memory; a GPU makes it much faster
Maintenance	You update, secure and monitor it yourself
Context length	Often shorter than hosted models unless configured

10.2 Common tools

Tool	What it is
Ollama	Command-line tool to download and run open models locally and expose a local API
LM Studio	Desktop app with a model catalogue, chat and a local server
Open WebUI	Browser chat interface that can sit on top of a local model server

Typical Ollama steps

```
ollama pull llama3.1            # download a model
ollama run llama3.1             # chat in the terminal
# local API (default port 11434)
curl http://localhost:11434/api/generate -d '{"model":"llama3.1","prompt":"Write 3 login test cases"}'
```

Choose a model that fits your memory. Smaller models (a few billion parameters) run on ordinary laptops; larger ones need substantial RAM or a GPU. Quantised versions use less memory with a small quality cost.

10.3 Customising a local model for your project

Method	What it is	When to use
System prompt / Modelfile	Fixed instructions and settings baked into a named model	Role, style, output format
RAG (retrieval-augmented generation)	Search your documents and give the relevant parts to the model with each question	Answers grounded in requirements, manuals, test cases
Fine-tuning	Further train the model on your examples	Rarely needed; costly; use only after prompting and RAG fall short

```
# Modelfile
FROM llama3.1
SYSTEM You are a senior QA engineer for an ERP contract system.
       Write concise test cases in table format. Do not invent requirements.
PARAMETER temperature 0.2

ollama create qa-assistant -f Modelfile
```

How RAG works

Your documents	Split into chunks + embeddings	Search for relevant chunks	Chunks + question to the model	Grounded answer

INSTRUCTOR TIP
Start with prompting, then add RAG over your requirement and test documents. That solves most project needs without fine-tuning.

ERP / YOUR PROJECT
An offline model is a good fit when contract data, pricing or counterparty information must not leave the company network.

SECTION 10 REVISION
- Local LLM = privacy, control, no per-token cost; trade-off is quality and hardware.
- Ollama or LM Studio to run; Modelfile or system prompt to customise.
- RAG grounds answers in your documents; fine-tuning is the last resort.

11. Generating Test Artifacts on the Fly

Lessons covered: Learning objective: generate test plan, test cases, test data and bug templates for given business requirements. Builds on sections 2 and 8.

11.1 The artifact pipeline

Business requirement	Test plan	Test cases	Test data	Execution	Bug reports

Feed each output into the next prompt, and keep the same requirement text and IDs throughout so everything stays traceable.

11.2 Worked example: requirement to artifacts

Requirement (sample): *'A user with the Approver role can approve a contract that is in Pending status. After approval the status becomes Approved and the Modify action is no longer available. A confirmation message must be shown.'*

Test cases generated

ID	Title	Steps	Expected	Type
TC01	Approve a pending contract	Log in as Approver; open Pending contract; Approve; Confirm	Status Approved; message shown	Positive
TC02	Modify not offered after approval	Open approved contract; check actions	Modify not available	Positive
TC03	User without Approver role	Log in as Creator; open Pending contract	Approve not offered	Negative
TC04	Cancel at confirmation dialog	Click Approve; choose No	Status stays Pending	Negative
TC05	Approve an already approved contract	Open approved contract; look for Approve	Not available	Negative

Test data generated

```
[
  {"testCaseId": "TC01", "user": "approver01", "contractStatus": "Pending"},
  {"testCaseId": "TC03", "user": "creator01",  "contractStatus": "Pending"},
  {"testCaseId": "TC05", "user": "approver01", "contractStatus": "Approved"}
]
```

11.3 Bug report template

Field	Content
Title	Short and specific: what fails where
Environment	Application version, URL, browser, user role
Preconditions	State and data required
Steps to reproduce	Numbered, exact
Expected result	From the requirement
Actual result	What happened, with message text
Severity and priority	Impact and urgency
Evidence	Screenshot, log, request/response
Requirement reference	ID or section

SAMPLE PROMPT
Turn these rough notes into a bug report using the template: Title, Environment, Preconditions, Steps, Expected, Actual, Severity, Evidence. Do not add facts that are not in my notes; list what is missing at the end. Notes: [paste].

11.4 Quality checks before you use any AI artifact

1. Does every test case trace to a requirement line?
2. Are expected results specific and verifiable?
3. Are there negative, boundary and role-based cases, not only happy paths?
4. Do field names, statuses and messages match the real application?
5. Is any invented rule listed under assumptions rather than treated as fact?
6. Has a second person reviewed it?

SECTION 11 REVISION
- Keep IDs and requirement text consistent through the pipeline.
- Ask for assumptions and missing information explicitly.
- Human review is the final gate for every artifact.

Appendix: Prompt Library, Glossary and Cheat Sheet

A. Reusable prompts

Task	Prompt
Test cases	Act as a senior QA engineer. From the requirement below write test cases using boundary value analysis, equivalence partitioning and state transition. Table: ID, Title, Preconditions, Steps, Expected, Type, Priority. Do not invent requirements; list assumptions.
Review a test	Review this automation code for locator stability, waits, assertion quality, duplication and hard-coded data. Report findings by severity with a suggested fix for each.
Failure analysis	Think step by step. Here is the failure log and the test code. List likely causes in order of probability, what evidence confirms each, and the fix for the most likely one.
Test data	Generate pairwise test data as JSON for these fields and options: [list]. Include boundary and invalid values. Synthetic data only.
Bug report	Convert these notes to a bug report with Title, Steps, Expected, Actual, Severity. Do not add facts. List missing information.
Coverage gaps	Compare these requirements with these test cases. Show a coverage matrix and list requirements with no or weak coverage, ranked by risk.

B. Glossary

Term	Meaning
LLM	Large language model that generates text and code
Token	Small unit of text the model processes; basis of cost and limits
Context window	Maximum tokens the model can consider at once
Zero-shot / few-shot	Prompting without / with examples
Chain of thought	Asking the model to reason step by step
Hallucination	Confident but incorrect or invented output
RAG	Retrieving your documents to ground the model's answer
MCP	Model Context Protocol; standard to connect AI to tools and data
Agent	Model with tools and instructions working in a loop toward a goal
Skill	Packaged instructions or knowledge an agent loads on demand
CLAUDE.md	Project memory file read at the start of a Claude Code session
Prompt injection	Malicious instructions hidden in content the AI reads
Self-healing	Automatic repair of broken locators by a tool
Shift-left	Testing earlier and at lower layers
POJO	Plain Java object used to map JSON
Webhook	URL that triggers a workflow when called
CI/CD	Automated build, test and delivery pipeline

C. One-page cheat sheet

Situation	Do this
Output is vague	Add context and a format constraint
Output format inconsistent	Add 2 to 3 examples (few-shot)
Reasoning task or failure analysis	Ask for step-by-step reasoning
Chat is getting long or confused	Summarise, start a new chat, reference files
Want AI to change code across files	Plan first, then Agent mode; review the diff
Need AI to use a real system	Use an MCP server with least privilege
Repeating the same instructions	Move them to custom instructions or a skill
Sensitive data involved	Synthetic data, enterprise tool or offline model
Tests keep failing	Classify: test, data, environment or application defect
Presenting to management	Pilot, measure, show a before and after

Final reminder: AI drafts, you decide. The value of a senior tester is judgement, and these tools give you more time to use it.